COA: Instruction Set - Complete Revision Notes

Compact exam notes: definitions, examples, classifications, RISC vs CISC, and instruction formats

1. Instruction Set - Basic Idea

Easy definition: An instruction set is the complete collection of instructions that a processor can recognize and execute. It is a part of the Instruction Set Architecture (ISA).

Examples of instruction mnemonics: **ADD, SUB, MOV, LOAD, STORE, JMP, IN, OUT, NOP**. The total number of instructions may range from fewer than 100 to several hundred. A larger instruction set does **not** automatically mean a more powerful processor.

Memory line: Instruction Set = All commands understood by the CPU.

2. Generic Classification of Instructions

Category	Purpose	Common examples
Data Movement	Move data among registers and memory	MOV, LOAD, STORE
Arithmetic & Logical	Perform calculations and bit/logic operations	ADD, SUB, MUL, DIV, AND, OR, XOR, SHIFT, CMP
Transfer / Control	Change normal program flow	JMP, JNZ, BZ, BNE, CALL, RETURN
Input / Output	Transfer data between CPU and I/O devices	IN, OUT, memory-mapped MOV/LOAD/STORE
Miscellaneous	Special processor/system operations	NOP, HALT, INT
Floating Point	Floating-point load/store/arithmetic	FLD, FST, FADD, FSUB, FMUL
BCD	Support decimal-oriented calculations	BCD-specific operations
Vector	Operate efficiently on vectors/matrices	Vector operations for graphics/matrix work
Emulated	User-defined/emulated operations on supporting CPUs	CPU-dependent

3. Data Movement Instructions

Easy definition: Data movement instructions transfer data from one location to another. They are among the most frequently used instructions.

Typical transfers: Register -> Register, Register -> Memory, Memory -> Register; some CPUs also support Memory -> Memory.

```
MOV R1, Total
MOV Total, R1
LOAD R1, Total
STORE R1, Total
```

Memory trick: LOAD = Memory to Register; STORE = Register to Memory.

4. Arithmetic and Logical Instructions

Easy definition: Arithmetic and logical instructions perform calculations, comparisons, and bit-level operations.

Arithmetic: ADD, SUB, MUL, DIV. **Logical:** AND, OR, XOR, SHIFT. Some CPUs also provide CMP (Compare).

```
ADD R1, TOTAL
ADD R1, R2, R3
XOR R4, TOTAL
MUL R4, MARKS
```

Logical instructions help implement conditions used in high-level constructs such as **if, for, and while**.

5. Condition Codes / Flags

After arithmetic or logical execution, condition-code flags may be updated to show the result state.

Flag	Meaning
Z - Zero	Result is zero

Flag	Meaning
S - Sign	Indicates sign of result
O - Overflow	Signed result exceeds representable range
C - Carry	Carry/borrow condition in arithmetic

Memory trick: ZSOC = Zero, Sign, Overflow, Carry.

6. Transfer / Control Instructions

Easy definition: Control-transfer instructions change the normal sequence of program execution.

```
JMP LABEL1 ; Jump
JNZ LABEL1 ; Jump if Not Zero
BZ LABEL2 ; Branch on Zero
BNE LABEL3 ; Branch on Not Equal
```

CALL transfers control to a subroutine; **RETURN** brings execution back to the calling program.

7. Input / Output Instructions

Easy definition: I/O instructions allow the CPU to exchange data with input/output devices.

I/O-mapped I/O: Uses separate I/O ports and dedicated instructions such as IN and OUT.
Memory-mapped I/O: I/O registers are assigned memory addresses, so ordinary data-movement instructions can access them.

```
IN Port#232
OUT Port#234
MOV R1, #FFEEEE
```

I/O-Mapped I/O	Memory-Mapped I/O
Uses separate I/O address/port space	I/O registers use memory address space
Uses dedicated instructions such as IN/OUT	Uses normal memory/data-movement instructions
Device accessed as a port	Device register appears as a memory address

8. Miscellaneous Instructions

Instruction	Easy meaning	Use
NOP	No Operation	Dummy/timing/alignment instruction
HALT	Stop execution	Brings processor/system to halt depending on design
INT	Interrupt	Invokes an interrupt/service mechanism

9. Floating Point, BCD, Vector and Emulated Instructions

Type	Easy definition / purpose	Examples
Floating Point	Special instructions for floating-point data	FLD, FST, FADD, FSUB, FMUL
BCD	Support decimal-number calculations efficiently	BCD-specific instructions
Vector	Operate on multiple related data elements; useful in graphics/matrices	Vector/matrix operations
Emulated	Operations supplied through emulation/user-defined support on a few CPUs	CPU-dependent

10. RISC and CISC

RISC = Reduced Instruction Set Computer. CISC = Complex Instruction Set Computer. Both are functionally complete; the main trade-off is simplicity of hardware decoding versus richness/compactness of instruction encoding.

Feature	RISC	CISC
Full form	Reduced Instruction Set Computer	Complex Instruction Set Computer
Instruction set	Fewer, simpler instructions	Larger, more complex instruction set
Instruction encoding	Generally fixed-length	Generally variable-length
Decoding	Simpler	More complex

Feature	RISC	CISC
Operand access	Registers heavily used; load/store approach common	Instructions may support richer memory operand forms
Program code	May require more instructions	May achieve a task with fewer instructions
Hardware design	Simpler control/decoding focus	More complex instruction decoding/control
Examples from notes	MIPS, ALPHA, PA-RISC, Intel i860, Motorola 88000, ARM	Intel x86, Motorola 68K, VAX

Memory trick: RISC = Reduced + Regular; CISC = Complex + Compact.

11. Instruction Set Format - Definition

Easy definition: Instruction format is the arrangement of bits/fields inside an instruction that tells the processor what operation to perform and where the operands are.

Field	Easy definition	Examples
Opcode	Specifies the operation	ADD, SUB, MOV
Operand(s)	Specifies data used by the operation	Register, memory address, immediate value
Addressing Mode	Tells how an operand/address is interpreted	Immediate, direct, indirect, indexed
Instruction Length	Total number of bits/bytes in the instruction	16-bit, 32-bit, 64-bit etc.

12. Key Considerations in Instruction Formats

• **Instruction length:** Shorter instructions save memory; longer instructions can encode more information.

• **Number of operands:** Zero to three operands are common and affect size/flexibility.

• **Addressing modes:** Determine how operands are accessed.

• **Ease of decoding:** Fixed-length is simpler to decode; variable-length is more compact but harder to decode.

13. Fixed-Length vs Variable-Length Instruction Format

Feature	Fixed-Length Format	Variable-Length Format
Typical association in notes	RISC	CISC
Instruction size	Every instruction occupies the same length	Length changes according to instruction/operands
Decoding	Simple and fast to decode	More complicated to decode
Memory use	May waste some encoding space	Uses only required space; compact
Example in notes	MIPS series	Motorola M6800 example

Easy definition - Fixed length: Every instruction has the same size.

Easy definition - Variable length: Instruction size changes depending on the operation and operand information.

14. Address-Based Instruction Formats

Instruction formats are also classified by the number of explicit operand/address fields: three-address, two-address, one-address, and zero-address.

Format	General form	Meaning / Example
Three-address	OP Dest, Src1, Src2	ADD R1, R2, R3 -> R1 = R2 + R3
Two-address	OP Dest/Src, Src	ADD R1, R2 -> R1 = R1 + R2
One-address	OP Address	Accumulator is implied: ADD A -> AC = AC + A
Zero-address	OP	Operands implied on stack: ADD uses top stack values

15. Three-Address Format

Easy definition: Three-address instructions explicitly specify two source operands and one destination.

```
ADD R1, R2, R3 ; R1 = R2 + R3
```

Format: **Opcode | Source1 | Source2 | Destination**

16. Two-Address Format

Easy definition: Two-address instructions specify two operands, and one operand also stores the result.

```
ADD R1, R2 ; R1 = R1 + R2
```

Format: **Opcode | Source/Destination | Source**

17. One-Address Format

Easy definition: One-address instructions explicitly specify one operand; the accumulator (AC) is usually implied.

```
ADD A ; AC = AC + A
```

Format: **Opcode | Address**

18. Zero-Address Format

Easy definition: Zero-address instructions do not explicitly name operands; operands are taken from the top of a stack.

```
ADD ; operands are obtained from stack top
```

Format: **Opcode only**

19. Solved Expression in 0/1/2/3-Address Formats

Expression: X = (A + B) * (C + D)

Format	Instruction sequence

Zero-address	`PUSH A`
	`PUSH B`
	`ADD`
	`PUSH C`
	`PUSH D`
	`ADD`
	`MUL`
	`POP X`

One-address	`LOAD A`
	`ADD B`
	`STORE T`
	`LOAD C`
	`ADD D`
	`MUL T`
	`STORE X`

Two-address	`MOV R1, A`
	`ADD R1, B`
	`MOV R2, C`
	`ADD R2, D`
	`MUL R1, R2`
	`MOV X, R1`

Three-address	`ADD R1, A, B`
	`ADD R2, C, D`
	`MUL X, R1, R2`

How to remember the four formats

Format	What is implied?	Quick memory
3-address	Nothing important; 3 fields are explicit	2 sources + 1 destination
2-address	Destination is also one source	Result overwrites one operand
1-address	Accumulator (AC)	AC does the hidden work
0-address	Stack operands	Top of stack does the hidden work

20. Ultra-Short Exam Definitions

Instruction Set: Collection of instructions supported by a CPU.

Data Movement Instruction: Instruction that transfers data between registers, memory, or I/O-related locations.

Arithmetic/Logical Instruction: Instruction that performs mathematical, comparison, or bitwise operations.

Control Instruction: Instruction that changes the normal flow of program execution.

I/O Instruction: Instruction used to communicate with input/output devices.

NOP: No Operation instruction.

HALT: Instruction used to stop execution.

RISC: CPU design with a reduced, simpler instruction set.

CISC: CPU design with a larger, more complex instruction set.

Instruction Format: Bit-field arrangement of an instruction.

Opcode: Field that specifies the operation to be performed.

Operand: Data, register, address, or value on which an operation acts.

Addressing Mode: Method used to interpret or locate an operand.

Fixed-Length Format: All instructions have the same size.

Variable-Length Format: Instruction size changes with operation/operand requirements.

Three-Address: Two source operands and one destination are explicit.

Two-Address: Two operands are explicit; one also stores the result.

One-Address: One operand is explicit; accumulator is implied.

Zero-Address: No explicit operand; operands are implied on the stack.

21. Last-Minute Revision Sheet

Classification: Data Movement -> Arithmetic/Logical -> Control -> I/O -> Miscellaneous -> Floating Point -> BCD -> Vector -> Emulated

Data movement: MOV, LOAD, STORE | **Arithmetic:** ADD, SUB, MUL, DIV | **Logical:** AND, OR, XOR, SHIFT

Control: JMP, JNZ, BZ, BNE, CALL, RETURN | **Misc:** NOP, HALT, INT

ZSOC: Zero, Sign, Overflow, Carry

RISC: fewer/simple, fixed-length, easier decoding | **CISC:** larger/complex, variable-length, compact encoding

Instruction format fields: Opcode + Operand(s) + Addressing Mode + Instruction Length

Address formats: 3-address = 2 source + destination; 2-address = one operand also destination; 1-address = accumulator; 0-address = stack

Golden memory line: RISC = Reduced/Regular; CISC = Complex/Compact; AC belongs to 1-address; Stack belongs to 0-address.